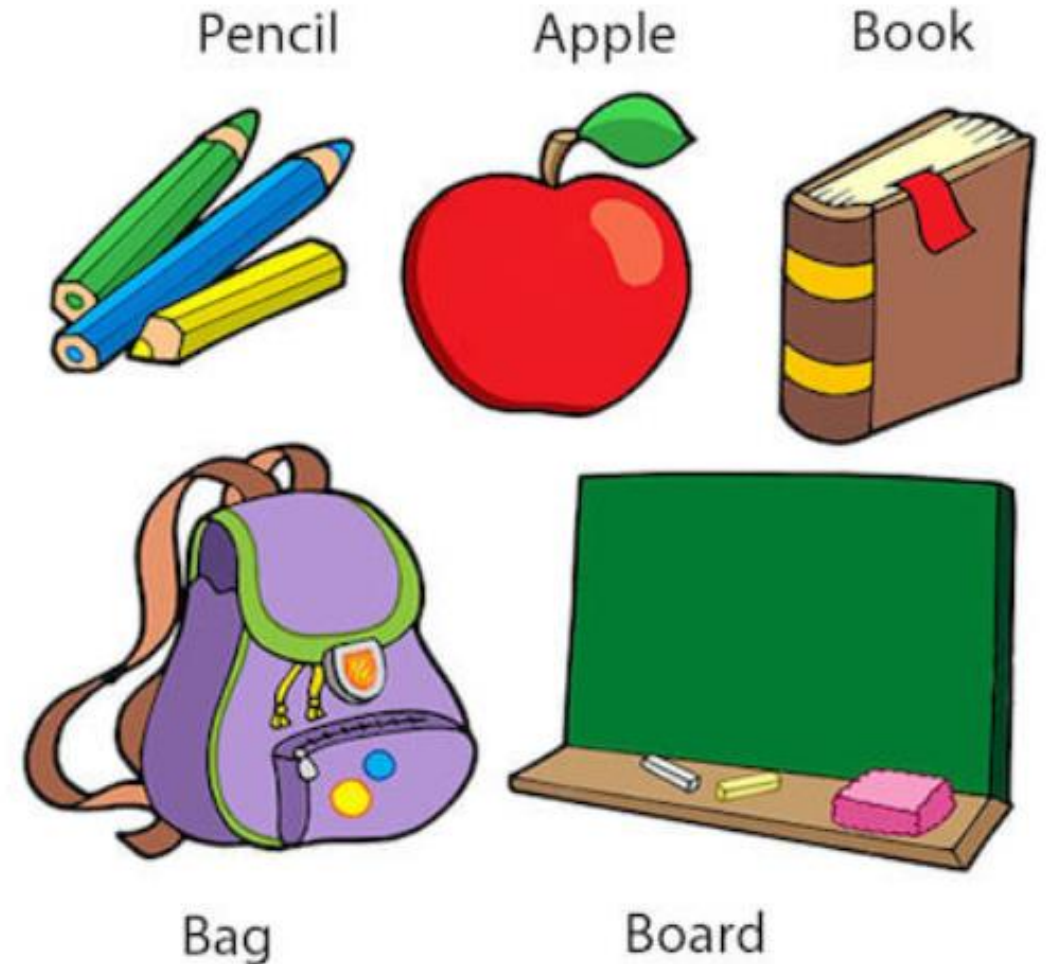

Object and class in java

Objects: Real World Examples

An entity that has state and behavior is known as an object
e.g., chair, bike, marker, pen,
table, car, etc. It can be physical
or logical (tangible and
intangible). The example of an
intangible object is the banking
system.

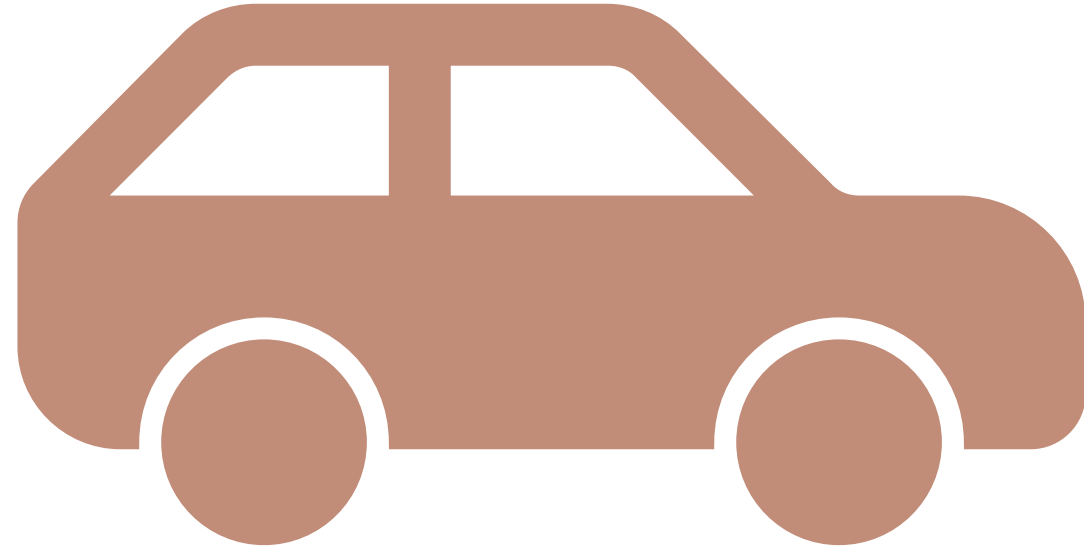


Object Characteristics:

State: An object has a state, represented by the **attributes of an object** and their values. For example, a Car object might have attributes like color, make, and currentSpeed.

Behavior: Objects have behaviors, which are **actions that an object can perform**, typically defined by methods in a class. For example, a Car might have behaviors such as `accelerate()` and `brake()`.

Identity: Each object has a **unique identity**, which allows objects to be **distinguished from each other**. Even if two objects have the same state, they are still considered distinct entities.

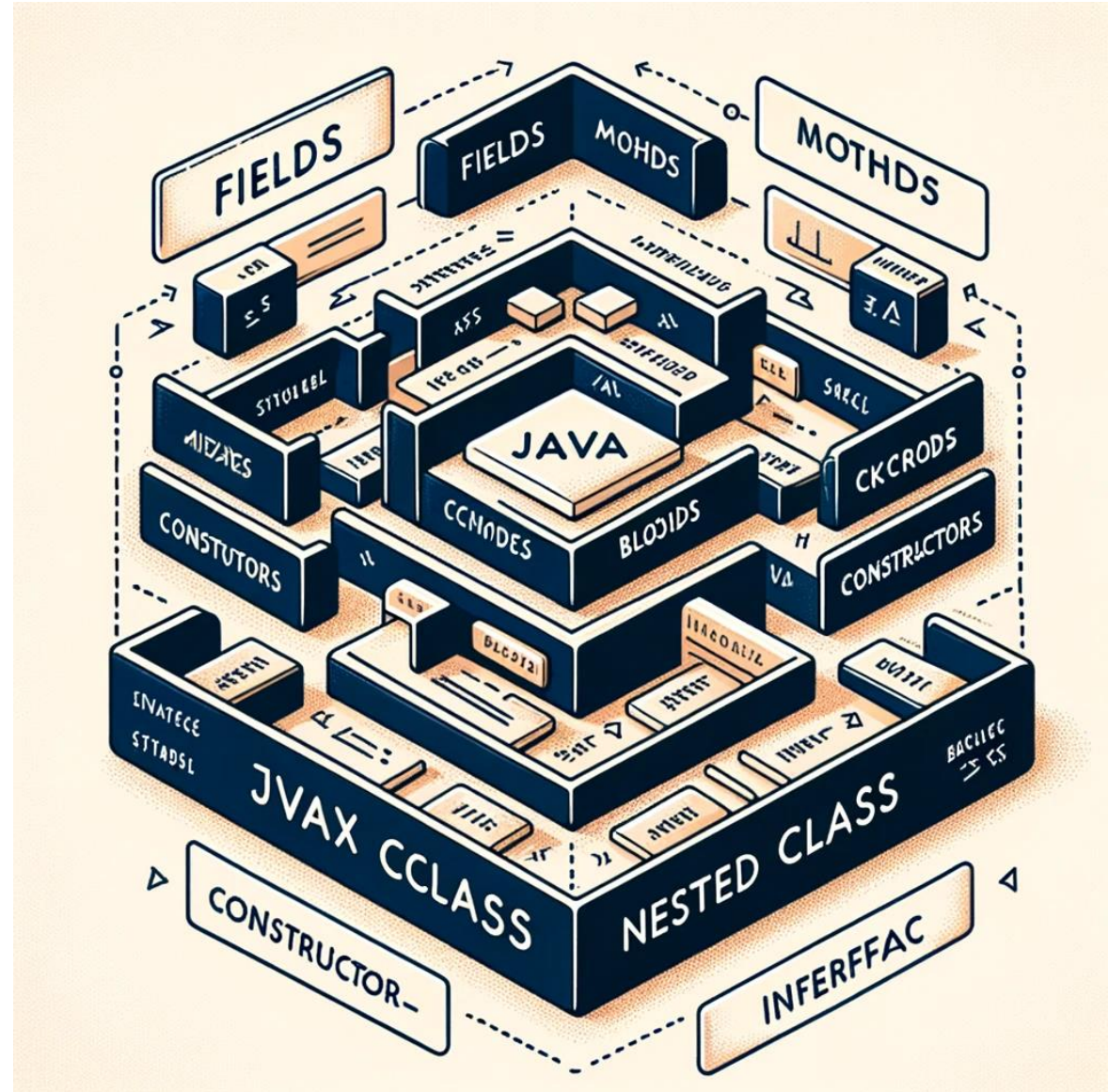


CLASS



A class in Java can contain:

- **Fields**
- **Methods**
- **Constructors**
- **Blocks**
- **Nested class and interface**



- 1.Fields:** Variables that store data or state for objects created from the class.
- 2.Methods:** Functions that define the behavior of objects created from the class. Methods can manipulate fields, perform operations, or interact with other objects.
- 3.Constructors:** Special methods that are called when an object is instantiated (created) from a class. Constructors initialize the new object's fields and perform any other startup procedures the object might require.
- 4.Blocks:** There are two types of blocks that can be included within a Java class:
 - Static blocks:** Code blocks that are executed only once when the class is loaded into memory. These are defined using the **static** keyword.
 - Instance initializer blocks:** Code blocks that are executed each time a new instance of the class is created. These blocks are used to perform operations or initializations that apply to all constructors in the class.
- 5.Nested Class and Interface:** A class can contain other classes or interfaces defined within its scope. These are known as nested classes and nested interfaces. Nested classes are divided into two categories:
 - Static nested classes:** Defined with the **static** keyword and do not have access to the enclosing class's instance variables or methods.
 - Inner classes:** Do not use the **static** keyword and have access to the enclosing class's instance variables and methods. Inner classes can be further categorized into local classes (defined within a block), anonymous classes (which have no name and are declared and instantiated all at once), and member classes (defined at the same level as instance variables).

It should start with the uppercase letter.

CLASS

Same for multiple words

This keyword is used to declare a class in Java. A class is a blueprint from which individual objects are created. In Java, every program must have at least one class.

Ex : Student

Ex : BranchName

SYNTAX TO DECLARE A CLASS

```
class <class_name> {  
    field;  
    method;  
}
```

SYNTAX FOR CREATING AN OBJECT

```
ClassName variableName = new ClassName();
```

PROGRAMMING EXAMPLES

```
object to mirror
mirror_mod.mirror_object =
operation == "MIRROR_X":
mirror_mod.use_x = True
mirror_mod.use_y = False
mirror_mod.use_z = False
operation == "MIRROR_Z":
mirror_mod.use_x = False
mirror_mod.use_y = False
mirror_mod.use_z = True

#selection at the end -add
mirror_ob.select= 1
modifier_ob.select=1
context.scene.objects.active
("Selected" + str(modifier_ob
mirror_ob.select = 0
= bpy.context.selected_object
data.objects[one.name].select
print("please select exactly

-- OPERATOR CLASSES --

types.Operator):
X mirror to the selected
object.mirror_mirror_x"
mirror_x"
```

Examples of Multiple Classes:

Consider a simple program that simulates a library. You might have a Book class, a Library class, and a Librarian class, each responsible for different aspects of the simulation.

// Class that represents a book

```
class Book {  
    String title;  
    String author;  
    // Constructor, getters, setters, and other methods...  
}
```

// Class that represents a library, which holds a collection of books

```
class Library {  
    List<Book> books;  
    // Methods to add books, check out books, and other library functions...  
}
```

// Class that represents a librarian, who manages the library operations

```
class Librarian {  
    String name;  
    Library library;  
    // Methods to assist library users, manage books, etc...  
}
```

Basic Object Creation

ClassName variableName = new ClassName();

class Book

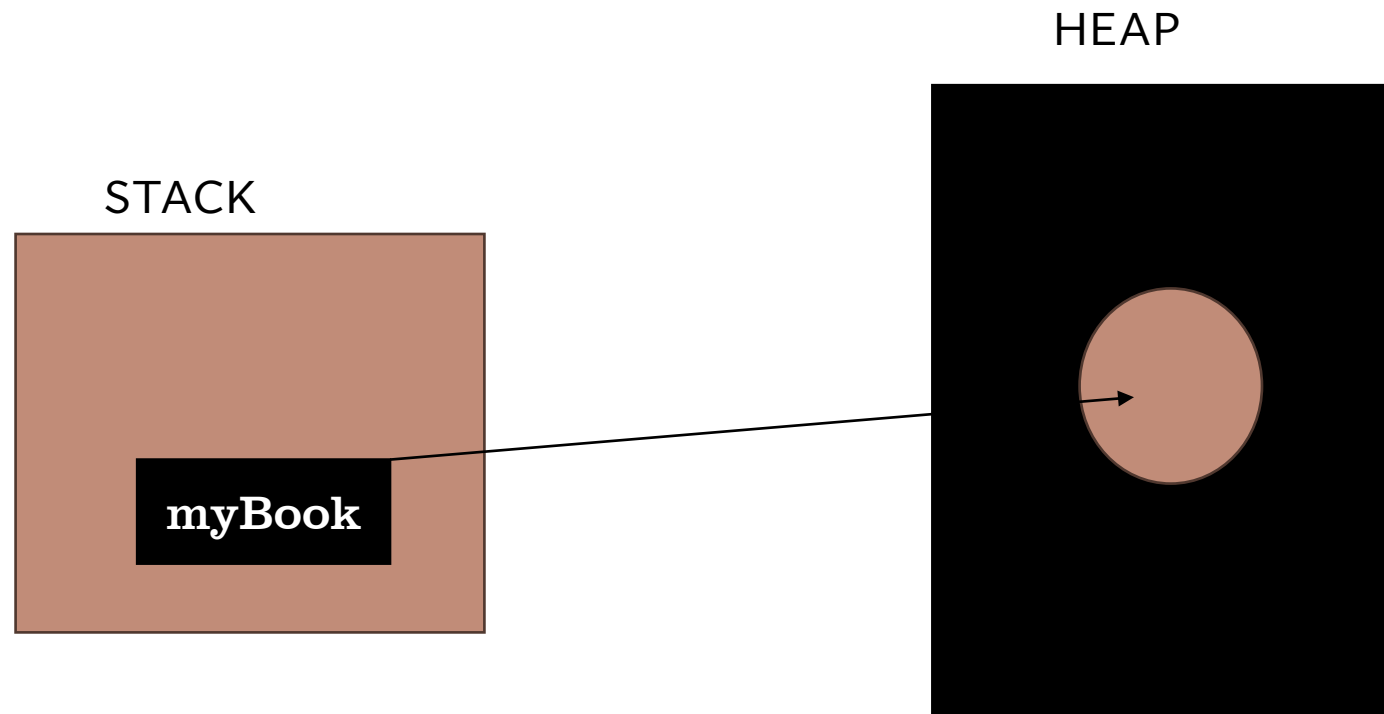
{

String title;

}

// Creating an object

Book myBook = new Book();



3 Ways to initialize object

There are 3 ways to initialize object in Java.

By reference variable

By method

By constructor

Object and Class Example: main within the class

In this example, we have created a Student class which has two data members id and name. We are creating the object of the Student class by new keyword and printing the object's value.

Here, we are creating a main() method inside the class.

//Java Program to illustrate how to define a class and fields

class Student{

//defining fields

int id;//field or data member or instance variable

String name;

//creating main method inside the Student class

public static void main(String args[]){

//Creating an object or instance

Student s1=new Student();//creating an object of Student

//Printing values of the object

System.out.println(s1.id);//accessing member through reference variable

System.out.println(s1.name); } }

OUTPUT:

0

null

Object and Class Example: main outside the class

In real time development, we create classes and use it from another class. It is a better approach than previous one. Let's see a simple example, where we are having main() method in another class.

//Java Program to demonstrate having the main method in another class

```
class Student{  
int id;  
String name;  
}
```

//Creating another class TestStudent1 which contains the main method

```
class TestStudent1 {  
public static void main(String args[]){  
Student s1=new Student();  
System.out.println(s1.id);  
System.out.println(s1.name);  
} }
```

OUTPUT:

0

null

Object and Class Example: Initialization through reference

Initializing an object means storing data into the object. Let's see a simple example where we are going to initialize the object through a reference variable.

```
class Student{  
    int id;  
    String name;  
}
```

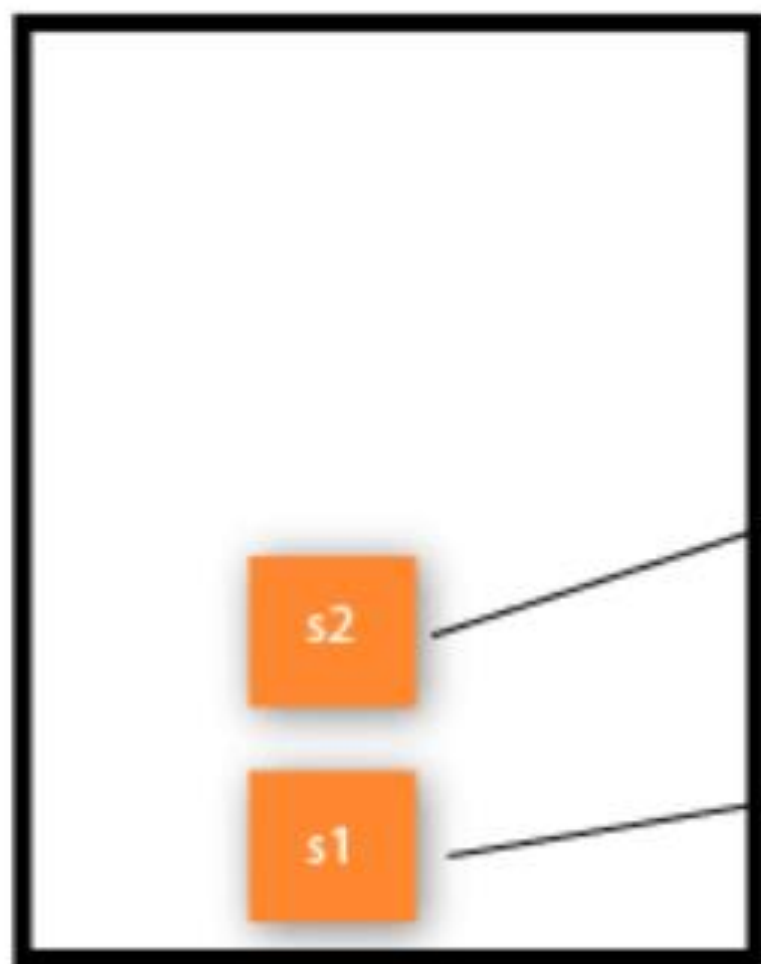
235550 Arun

```
class TestStudent2{  
    public static void main(String args[]){  
        Student s1=new Student();  
        s1.id=235550;  
        s1.name="Arun";  
        System.out.println(s1.id+" "+s1.name);  
        //printing members with a  
        white space    } }  
_____
```

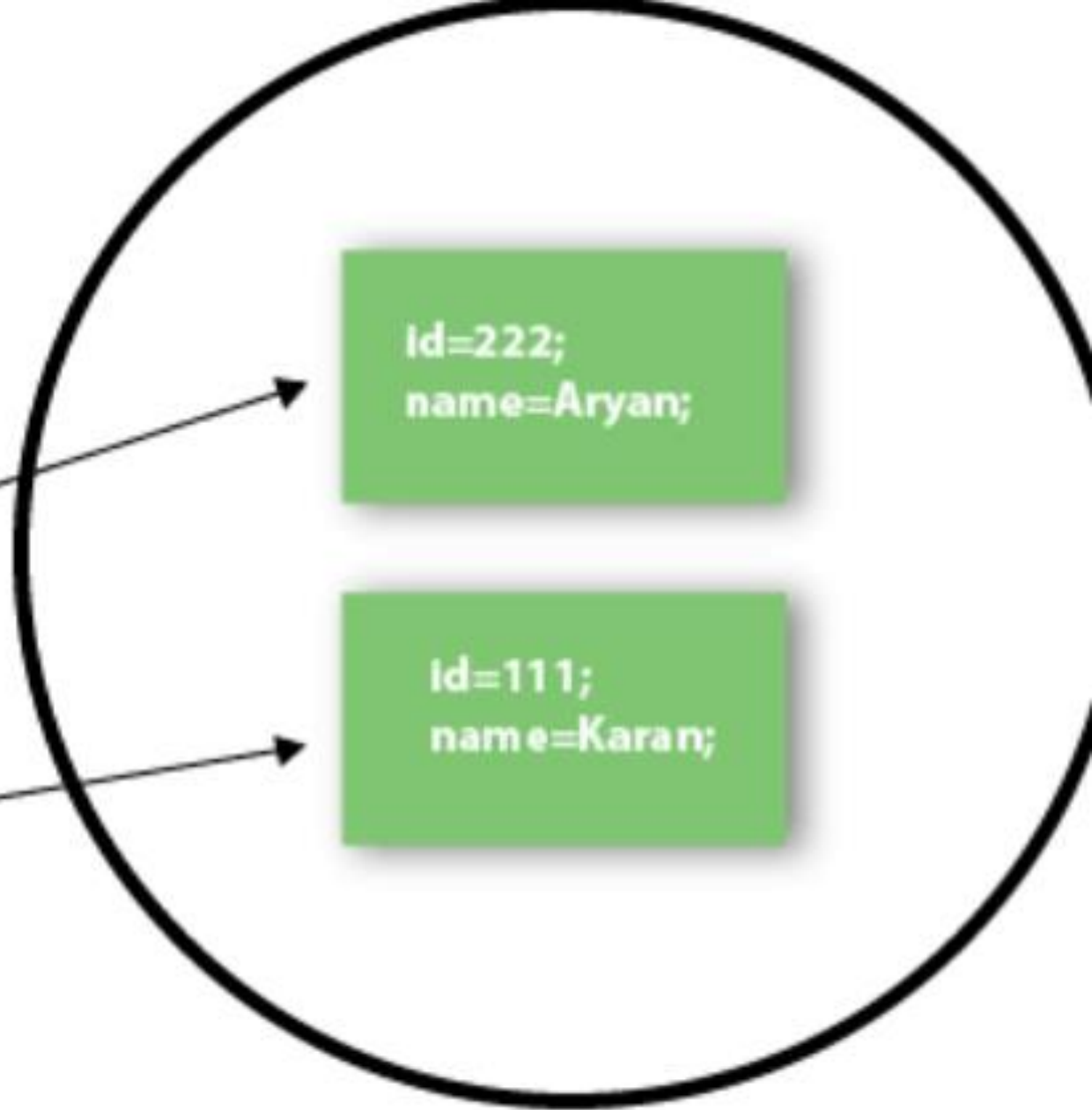
Object and Class Example: Initialization through method

```
class Student{
    int rollno;
    String name;
    void insertRecord(int r, String n){
        rollno=r;
        name=n;
    }
    void displayInformation(){System.out.println(rollno+" "+name);}
}

class TestStudent4{
    public static void main(String args[]){
        Student s1=new Student();
        Student s2=new Student();
        s1.insertRecord(111,"Karan");
        s2.insertRecord(222,"Aryan");
        s1.displayInformation();
        s2.displayInformation(); } }
```



Stack Memory



Heap Memory

Object and Class Example: Initialization through a constructor

```
class Employee{
    int id;
    String name;
    float salary;
    void insert(int i, String n, float s) {
        id=i;
        name=n;
        salary=s;
    }
    void display(){System.out.println(id+" "+name+" "+salary);}
}

public class TestEmployee {
    public static void main(String[] args) {
        Employee e1=new Employee();
        Employee e2=new Employee();
        Employee e3=new Employee();
        e1.insert(101,"ajeet",45000);
        e2.insert(102,"irfan",25000);
        e3.insert(103,"nakul",55000);
        e1.display();
        e2.display();
        e3.display();    } }
```

Ways to Create Object in Java



01

*By new
keyword*

02

*By newInstance()
method*

03

*By clone()
method*

04

By deserialization

05

*By factory
method*

Using new Keyword

Using the new keyword is the most popular way to create an object or instance of the class. When we create an instance of the class by using the new keyword, it allocates memory (heap) for the newly created object and also returns the reference of that object to that memory. The new keyword is also used to create an array. The syntax for creating an object is:

```
ClassName object = new ClassName();
```

Using clone() Method

The clone() method is the method of Object class. It creates a copy of an object and returns the same copy. The JVM creates a new object when the clone() method is invoked. It copies all the content of the previously created object into new one object. Note that it does not call any constructor.

We must implement the Cloneable interface while using the clone() method. The method throws CloneNotSupportedException exception if the object's class does not support the Cloneable interface. The subclasses that override the clone() method can throw an exception if an instance cannot be cloned.

SYNTAX:

We use the following statement to create a new object.

```
ClassName newobject = (ClassName) oldobject.clone();
```

Using newInstance() Method of Class class

The newInstance() method of the Class class is also used to create an object. It calls the default constructor to create the object. It returns a newly created instance of the class represented by the object. It internally uses the newInstance() method of the Constructor class.

We can create an object in the following ways:

```
ClassName object = ClassName.class.newInstance();
```

Using newInstance() Method of Constructor class.

It is similar to the newInstance() method of the Class class. It is known as a reflective way to create objects. The method is defined in the Constructor class which is the class of java.lang.reflect package. We can also call the parameterized constructor and private constructor by using the newInstance() method. It is widely preferred in comparison to newInstance() method of the Class class.

```
Constructor<Employee> constructor = Employee.class.getConstructor();  
Employee emp3 = constructor.newInstance();
```

4

Using Deserialization

In Java, serialization is the process of converting an object into a sequence of byte-stream. The reverse process (byte-stream to object) of serialization is called deserialization. The JVM creates a new object when we serialize or deserialize an object.

It does not use constructor to create an object. While using deserialization, the Serializable interface (marker interface) must be implemented in the class.

```
ObjectInputStream in = new ObjectInputStream(new  
FileInputStream("objectData.ser"));  
MyClass obj = (MyClass) in.readObject();
```



Java String

What is String in Java?

Generally, String is a sequence of characters. But in Java, string is an object that represents a sequence of characters. The `java.lang.String` class is used to create a string object.

How to create a string object?

There are two ways to create String object:

- ❖ By string literal

- ❖ By new keyword

1) String Literal

Java String literal is created by using double quotes. For Example:

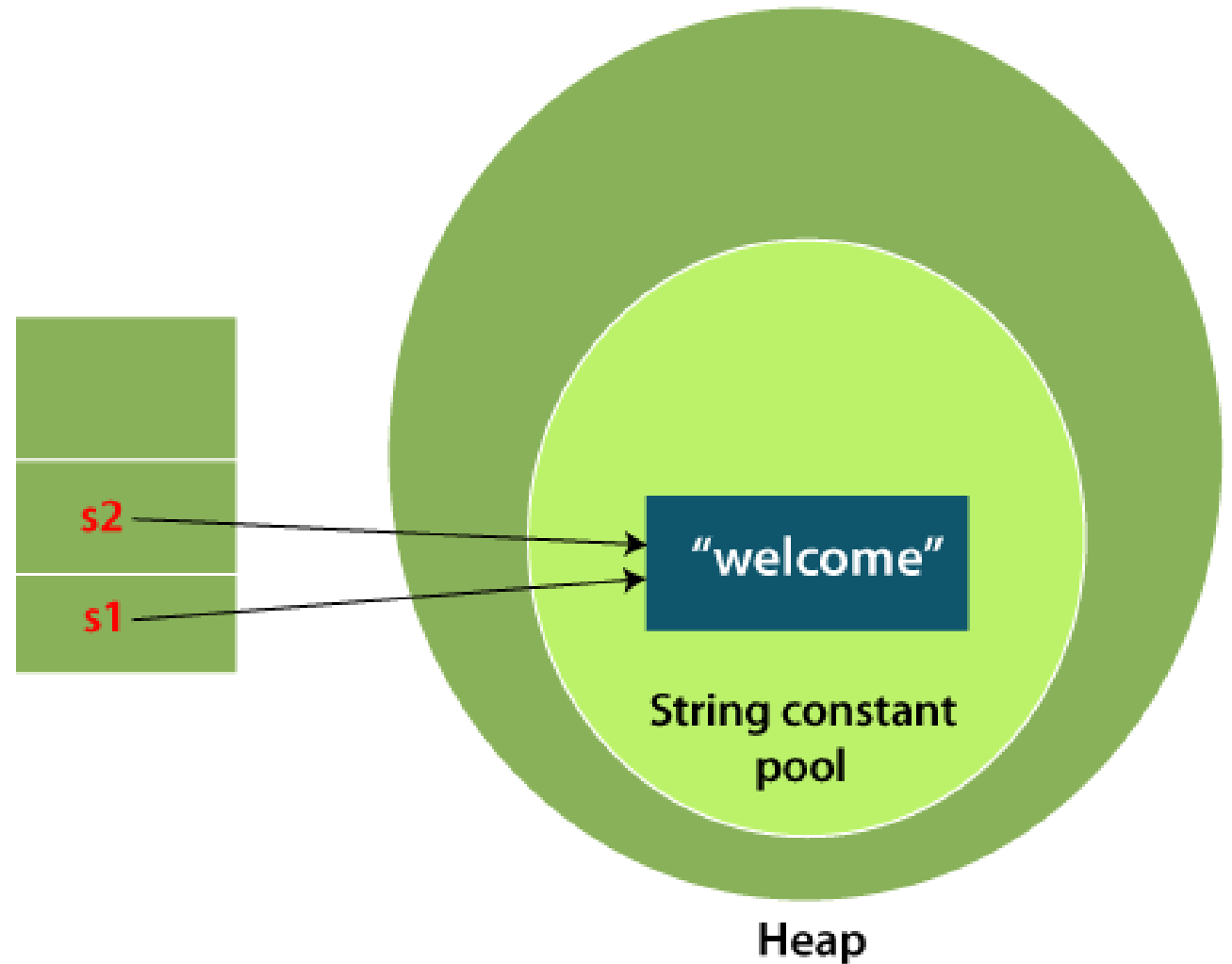
```
String s="welcome";
```

Each time you create a string literal, the JVM checks the "string constant pool" first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new string instance is created and placed in the pool. For example:

```
String s1="Welcome";
```

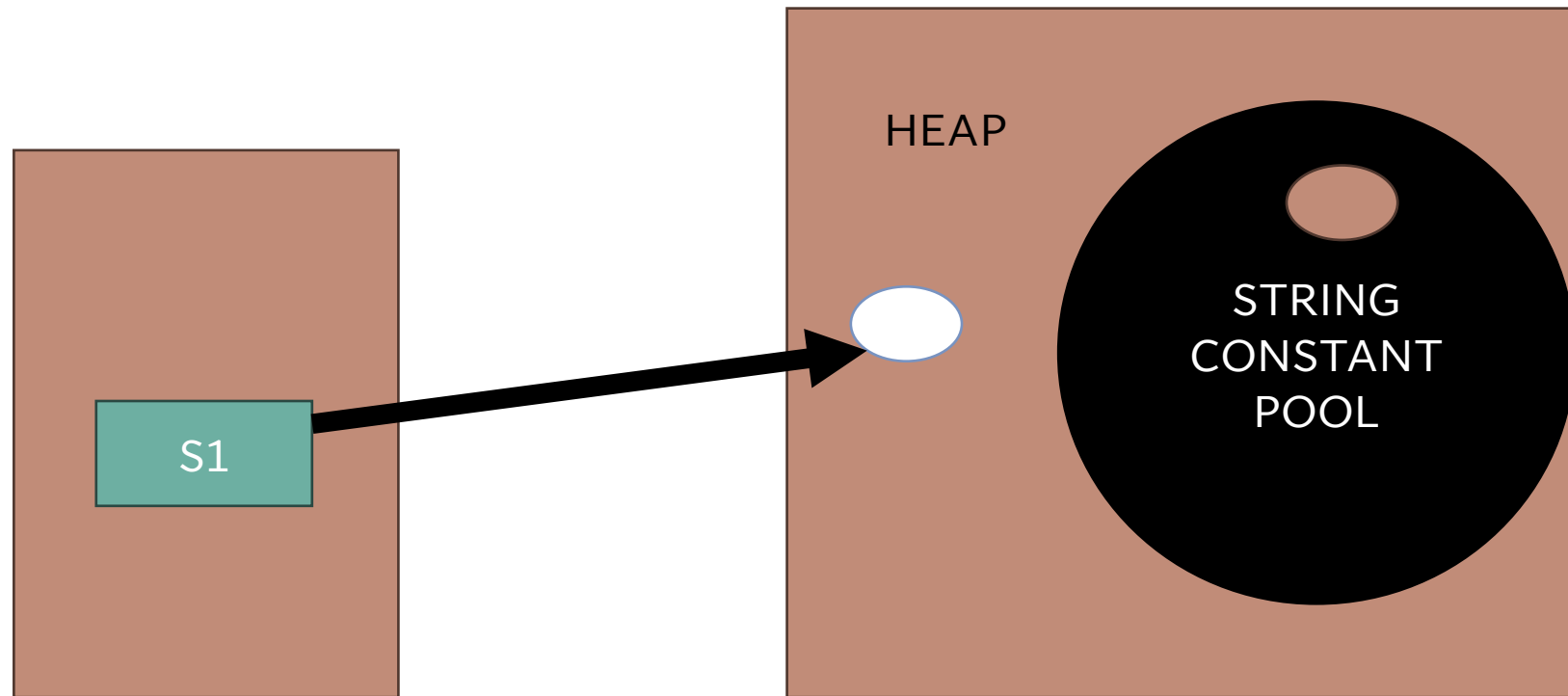
```
String s2="Welcome";//It doesn't create a new instance
```

- In the above example, only one object will be created. Firstly, JVM will not find any string object with the value "Welcome" in string constant pool that is why it will create a new object. After that it will find the string with the value "Welcome" in the pool, it will not create a new object but will return the reference to the same instance.



2) By new keyword

**String s=new String("Welcome");
//creates two objects and one reference variable**



// Define a class named SimpleString

public class SimpleString {

// The main method - entry point of the Java program

public static void main(String[] args) {

// Declare a String variable named message and initialize it

String message = "Hello, Java!";

// Print the value of the variable message to the console

System.out.println(message);

}

}

String str = new String("Hello");

OR

String str = "Hello";

Available in java.lang.* package – this is the default package so we no need to write explicitly in the program.

Disadvantages:

- **Strings class objects are immutable(can not modify on same location)** – means once the string is intialized we cannot modify the string unless creating a new string.
 - **Ex: String str = new String("Hello");** - for this object one reference will be allocated. While we trying to modify this one that can not be done in the same location.
 - If we are modifying the same string one more reference will be created in the heap memory.
 - So once the object is created that object reference will be in heap memory. So how many time the user modifies the string that no of times the reference will be added to the heap memory.
-

❑ **String str = new String("Hello");**

- ❑ **Length()** – returns the length of the string – **str.length()**
- ❑ **indexOf()** – Returns the index value of first character in given string - **indexOf(str)**
- ❑ **charAt()** – Returns the character at given index. – **str.charAt(index)**
- ❑ **Replace()** – Replaces the old string with new string – **str.replace(old string,new string)**
- ❑ **toLowerCase()** - Everything will be converted into lowercase character. – **str.toLowerCase()**
- ❑ **toUpperCase()** - Everything will be converted into uppercase character – **str.toUpperCase()**
- ❑ **compareTo()** – compare two strings – **it will returns 0 if it is equal. Returns +ve** if first mismatch of first string > first mismatch of second string
- ❑ **Returns -ve** if first mismatch of first string < first mismatch of second string – **ex Hello = hello**
compare the Unicode of first string H and h. **Unicode(H) – Unicode(h)** if value is > it will return +ve value
- ❑ **trim()** – Remove all the whitespaces which appear before and after the string. Ex – str = “ Hello “;
- ❑ **concat()** – **str.concat(str)** – it will concatenate the string – **str.concat(str)** – but it will be not saved in a same location. If we assign this string to new string then the concatenate will be applied ex :

str2 = str.concat("Welcome"); it will print hello welcome(modification is done on another object.

No.	Method	Description
1	<u><a>char charAt(int index)</u>	It returns char value for the particular index
2	<u><a>int length()</u>	It returns string length
3	<u><a>static String format(String format, Object... args)</u>	It returns a formatted string.
4	<u><a>static String format(Locale l, String format, Object... args)</u>	It returns formatted string with given locale.
5	<u><a>String substring(int beginIndex)</u>	It returns substring for given begin index.
6	<u><a>String substring(int beginIndex, int endIndex)</u>	It returns substring for given begin index and end index.
7	<u><a>boolean contains(CharSequence s)</u>	It returns true or false after matching the sequence of char value.

8	<u>static String join(CharSequence delimiter, CharSequence... elements)</u>	It returns a joined string.
9	<u>static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)</u>	It returns a joined string.
10	<u>boolean equals(Object another)</u>	It checks the equality of string with the given object.
11	<u>boolean isEmpty()</u>	It checks if string is empty.
12	<u>String concat(String str)</u>	It concatenates the specified string.
13	<u>String replace(char old, char new)</u>	It replaces all occurrences of the specified char value.
14	<u>String replace(CharSequence old, CharSequence new)</u>	It replaces all occurrences of the specified CharSequence.

length()

Returns the length of a string (the number of characters in the string).

```
String str = "VITAP";  
System.out.println(str.length());    // Outputs 5
```

charAt(int index)

Returns the character at the specified index.

```
String str = "VITAP";  
System.out.println(str.charAt(1));  
  
// Outputs 'I'
```

substring(int beginIndex, int endIndex)

Returns a substring from the specified beginIndex to endIndex-1.

```
String str = "Hello";  
System.out.println(str.substring(1, 3));  
// Outputs "el"
```

concat(String str)

Concatenates the specified string to the end of the calling string.

```
String str1 = "Vit";  
String str2 = " Ap";  
System.out.println(str1.concat(str2));
```

// Outputs "Vit Ap"

equals(Object anotherObject) :

Compares the calling string to the specified object. The result is true if and only if the argument is not null and is a String object that represents the same sequence of characters as this object.

```
String str1 = "Hello";           // Outputs true
String str2 = "Hello";
System.out.println(str1.equals(str2));
```

equalsIgnoreCase(String anotherString):

Compares the calling string to another string, ignoring case considerations.

```
String str1 = "hello";
String str2 = "HELLO";
System.out.println(str1.equalsIgnoreCase(str2)); // Outputs true
```

toLowerCase() and toUpperCase()

Converts all the characters in the string to lower case or upper case.

```
String str = "Hello World";  
System.out.println(str.toLowerCase()); // Outputs "hello world"  
System.out.println(str.toUpperCase()); // Outputs "HELLO WORLD"
```

trim()

Returns a copy of the string, with leading and trailing whitespace omitted.

```
String str = " Hello World ";  
System.out.println(str.trim()); // Outputs "Hello World"
```

replace(char oldChar, char newChar)

Returns a new string resulting from replacing all occurrences of oldChar in this string with newChar.

String str = "Hello World";

System.out.println(str.replace('l', 'p')); // Outputs "Heppo Worp"

```
class StringDemo
```

```
{
```

```
    public static void main(String arg[])
```

```
    {
```

```
        String str = new String(" Hello ");
```

```
        System.out.println(str);
```

```
        System.out.println(str.length());
```

```
        System.out.println(str.indexOf("H"));
```

```
        System.out.println("Character at="+str.charAt(4));
```

```
        System.out.println(str.toUpperCase());
```

```
        System.out.println(str.compareTo("Hello"));
```

```
        System.out.println(str.trim());
```

```
    }
```

```
}
```
